

Paper: Comparative Analysis and Replication of Machine Learning Models for Earthquake Magnitude Prediction

Results Reported in the Original Paper

The original paper reports the following results:

Model	MAE	RMSE	R ²	CV-RMSE
Random Forest	0.217051	0.322398	0.574261	0.317599
SVM	0.219773	0.338342	0.531113	0.331371
Decision Tree	0.282226	0.435265	0.223995	0.433322
Neural Network	0.237543	0.429644	0.243907	0.337684

The paper concludes that Random Forest is the best-performing model because it achieved the lowest MAE and RMSE and the highest R². SVM was the second-best model.

Results Obtained From the Implemented Notebook

The notebook produced the following results:

Model	MAE	RMSE	R ²	CV-RMSE
Random Forest	0.269857	0.380107	0.216678	0.400061
SVM	0.280251	0.421845	0.035206	0.416958
Decision Tree	0.347406	0.510227	-0.411419	0.523573
Neural Network	0.367850	0.520937	-0.471292	0.737596

The notebook shows that Random Forest currently performs best among the four implemented models.

The Decision Tree, SVM, and Neural Network results are also directly reported in the notebook.

Comparison Between the Original Paper and the Replication

The comparison shows that the **ranking of the models is partly different in magnitude but the Random Forest remains the strongest model in the current implementation.**

1. Random Forest

The paper reports an R^2 of **0.574261**, whereas the current implementation obtains **0.216678**.

The Random Forest result is therefore substantially weaker than the published result.

However, the current Random Forest still performs better than the other three models in the implemented experiment.

2. SVM

The paper reports an R^2 of **0.531113**, while the notebook obtains only **0.035206**.

This is a substantial difference.

Because SVM is highly dependent on feature scaling, the preprocessing difference is particularly important when interpreting this result.

3. Decision Tree

The paper reports:

$$R^2 = 0.223995$$

while the notebook obtains:

$$R^2 = -0.411419$$

A negative R^2 means that, on the current test set, the model performs worse than simply predicting the mean target value.

Therefore, the current Decision Tree implementation does not reproduce the performance reported in the paper.

4. Neural Network

The paper reports:

$$R^2 = 0.243907$$

while the notebook obtains:

$$R^2 = -0.471292$$

The Neural Network therefore performs considerably worse in the current implementation.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVR
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score
import warnings
warnings.filterwarnings('ignore')
```

```
df = pd.read_csv("database.csv")
```

```
num_fill_cols = [
    'Azimuthal Gap',
    'Horizontal Distance',
    'Root Mean Square',
    'Horizontal Error',
    'Depth Error',
    'Magnitude Error',
    'Magnitude Seismic Stations',
    'Depth Seismic Stations',
]
```

```
for c in num_fill_cols:
    df[c] = df[c].fillna(df[c].mean())
```

```
df['Magnitude Type'] = df['Magnitude Type'].fillna(df['Magnitude Type'].mode()[0])
```

```
dfCleaned = pd.get_dummies(
    df,
    columns=['Magnitude Type', 'Source', 'Type', 'Status',
             'Location Source', 'Magnitude Source'],
    drop_first=True
)
```

```
bool_cols = dfCleaned.select_dtypes(include=['bool']).columns
dfCleaned[bool_cols] = dfCleaned[bool_cols].astype(int)
dfCleaned = dfCleaned.drop(['ID', 'Date', 'Time'], axis=1)
```

```
scale_cols = ['Latitude', 'Longitude', 'Depth', 'Azimuthal Gap',
              'Horizontal Distance', 'Root Mean Square', 'Horizontal Error',
              'Depth Error']
scaler = StandardScaler()
```

```
x = dfCleaned.drop('Magnitude', axis=1)
y = dfCleaned['Magnitude']
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)
```

```
dfCleaned[scale_cols] = scaler.fit_transform(dfCleaned[scale_cols])
```

```
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
```

```
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
cv_rmse = np.sqrt(-cross_val_score(
    model, x, y, cv=5, scoring='neg_mean_squared_error'
)).mean()
```

```
print(f"Random Forest -> MAE: {mae:.6f} RMSE: {rmse:.6f} R²: {r2:.6f} CV_RMSE: {cv_rmse:.6f}")
```

```
Random Forest -> MAE: 0.269857 RMSE: 0.380107 R²: 0.216678 CV_RMSE: 0.400061
```

```
dt_model = DecisionTreeRegressor(  
    criterion='squared_error',  
    splitter='best',  
    max_depth=None,  
    min_samples_split=2,  
    min_samples_leaf=1,  
    random_state=42  
)
```

```
dt_model.fit(x_train, y_train)  
y_pred_dt = dt_model.predict(x_test)
```

```
mae_dt = mean_absolute_error(y_test, y_pred_dt)  
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))  
r2_dt = r2_score(y_test, y_pred_dt)  
cv_rmse_dt = np.sqrt(-cross_val_score(  
    dt_model, x, y, cv=5, scoring='neg_mean_squared_error'  
)).mean()
```

```
print(f"Decision Tree -> MAE: {mae_dt:.6f} RMSE: {rmse_dt:.6f} R²: {r2_dt:.6f} CV_RMSE: {cv_rmse_dt:.6f}")
```

```
Decision Tree -> MAE: 0.347406 RMSE: 0.510227 R²: -0.411419 CV_RMSE: 0.523573
```

```
svm_model = SVR(kernel='rbf', C=1.0, epsilon=0.1, gamma='scale')  
svm_model.fit(x_train, y_train)  
y_pred_svm = svm_model.predict(x_test)
```

```
mae_svm = mean_absolute_error(y_test, y_pred_svm)  
rmse_svm = np.sqrt(mean_squared_error(y_test, y_pred_svm))  
r2_svm = r2_score(y_test, y_pred_svm)  
cv_rmse_svm = np.sqrt(-cross_val_score(  
    svm_model, x, y, cv=5, scoring='neg_mean_squared_error'  
)).mean()
```

```
print(f"SVM -> MAE: {mae_svm:.6f} RMSE: {rmse_svm:.6f} R²: {r2_svm:.6f} CV_RMSE: {cv_rmse_svm:.6f}")
```

```
SVM -> MAE: 0.280251 RMSE: 0.421845 R²: 0.035206 CV_RMSE: 0.416958
```

```
nn_model = MLPRegressor(  
    hidden_layer_sizes=(100,),  
    activation='relu',  
    solver='adam',  
    alpha=0.0001,  
    learning_rate='adaptive',  
    max_iter=1000,  
    random_state=42  
)
```

```
nn_model.fit(x_train, y_train)  
y_pred_nn = nn_model.predict(x_test)  
mae_nn = mean_absolute_error(y_test, y_pred_nn)  
rmse_nn = np.sqrt(mean_squared_error(y_test, y_pred_nn))  
r2_nn = r2_score(y_test, y_pred_nn)  
cv_rmse_nn = np.sqrt(-cross_val_score(  
    nn_model, x, y, cv=5, scoring='neg_mean_squared_error'  
)).mean()
```

```
print(f"Neural Network -> MAE: {mae_nn:.6f} RMSE: {rmse_nn:.6f} R²: {r2_nn:.6f} CV_RMSE: {cv_rmse_nn:.6f}")
```

```
Neural Network -> MAE: 0.367850 RMSE: 0.520937 R²: -0.471292 CV_RMSE: 0.737596
```

